



江西科技师范大学

Jiangxi Science and Technology Normal University

本科生毕业论文（设计）

题目（中文）：智能垃圾分类督导系统设计与开发

（外文）：Design and Development of Intelligent

Garbage Classification Supervision System

学 院：数学科学学院

年级专业：22 级信息管理与信息系统

学生姓名：杨圣洲 学号：20224729

指导教师：刘庆华 职称：讲师

2026 年 5 月

成绩:

江西科技师范大学

毕业论文（设计）

题目（中文）：智能垃圾分类督导系统设计与开发

（外文）：Design and Development of Intelligent
Garbage Classification Supervision System

学 院：数学科学学院

年级专业：22 级信息管理与信息系统

学生姓名：杨圣洲 学号：20224729

指导教师：刘庆华 职称：讲师

2026 年 5 月

毕业论文诚信声明

本人郑重声明：

所提交的毕业论文《智能垃圾分类督导系统设计与开发》是本人在
指导老师的指导下，独立研究、写作的成果。论文中所引用是他人的无论以何
种方式发布的文字、研究成果，均在论文中以明确方式标明。

本声明的法律结果由本人独自承担。

毕业论文作者签名：杨圣洲

2026 年 5 月 20 日

摘要

针对社区垃圾分类督导效率低下、居民参与度不高、数据链路不完整、缺乏激励和引导等问题，采取前后端分离的多服务分布式架构，采用人工智能+物联网的方式创建起一个智能垃圾分类督导系统。系统可以实现对设备的自动识别、分类、积分奖励管理，也可以查看到每个设备的具体运行状况，出现满溢问题的时候，会发出相应的警报信息，并发送清运指令。经过测试可知，在 50 用户的并发登录情况下的系统平均响应时间小于 200ms，垃圾的识别准确率达到了 78%，认证的稳定、权限控制的有效，设备告警灵敏、调度闭环顺畅。本研究给社区垃圾分类智能化管理提供了一条可行之路，也给予实践上的借鉴。创新点有创建边缘云协同的垃圾分类轻量化识别架构，用 YOLOv8s 做边缘端实时识别和云端数据迭代，提出精细化的分层积分激励机制，达成基础积分，准确率奖励，连续投放激励的差别引导，达成物联网驱动的设备数字化运维，形成心跳保活，异常自动上报，满溢预警联动机制。

关键词：垃圾分类；智能督导；YOLOv8；Spring Boot；积分激励

Abstract

To address problems including insufficient efficiency for overseeing the community garbage sorting, not many people desire to participate and there's scattered information without sufficient incentives & directions, on top of that with the help of front-end services and AI&IoT we create an IntelligentGarbageClassificationSupervision system System has realized some of the roles like basic role of an artificial Intelligence with respect to Intelligent Recognition and Classification Identification. Precise point improvement in incentives, timely equipment status monitoring, overflow alarm for garbage transport if it's happening now. To monitor and manage every action related to discharging garbage. The result shows that when 50 people are online at the same time, the average response time of our system is less than 200ms. Garbage classification rate was 78%, authentication stable, correct permission control, point counting and concurrent safety have achieved, alarm to sensitive equipment, loop smooth Schedule are all possible. And it gives out a possible plan and actual reference to realize the smart management of community garbage classification. Innovative Points: The edge-cloud cooperation lightweight recognition framework for realization of garbage classification creation, realizing real-time end-edge image prediction and continuous update of local end's predicted objects' information by YOLOV8S model; improving hierarchical point system that is able deliver any type instruction about basic points, precise points and endure disposal incentive 3 establishing IOT based digital O&M systems so as to realize iot o&m heartbeat keeping alive mechanism, auto anomaly reporting, detecting the blockages.

Key Words: Waste Classification; Intelligent Supervision; YOLOv8; Spring Boot; Point Incentive

目录

第一章 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	1
1.3 研究内容与目标	2
1.4 论文组织结构	2
第二章 相关技术综述	3
2.1 前端及移动端开发技术	3
2.2 后端及数据库技术	3
2.3 AI 识别与物联网技术	4
第三章 系统需求分析	6
3.1 系统总体概述	6
3.2 功能需求分析	7
3.3 非功能需求分析	8
3.4 系统可行性研究	8
第四章 系统设计	10
4.1 系统架构设计	10
4.2 核心机制设计	11
4.3 数据库设计	13
4.4 系统接口规范	19
第五章 系统实现	23
5.1 后端实现	23
5.2 前端实现	26
5.3 部署实现	28
第六章 系统测试	30
6.1 测试环境与策略	30
6.2 功能测试	30
6.3 性能测试与准确率评估	34
6.4 测试结果分析	35
第七章 总结与展望	37

7.1 工作总结	37
7.2 存在问题	37
7.3 未来展望	37
参考文献	39
致谢	40

第一章 绪论

1.1 研究背景与意义

我国城市化建设进入中期之后，垃圾分类成了智慧城市治理的刚需。生态环境部统计表明，2024 年我国城市生活垃圾清运量为 2.62 亿吨。就如此大一个规模，如果源头不进行分类、混装混运的话，一方面会降低垃圾的资源利用率，另一方面也大大增加了末端处理的成本。

基层社区存在如下明显的问题，居民的垃圾分类意识差，主动性不高，且系统缺少数据闭环。只有前端分类细致，后端处理效率才能提高。依据上述问题，本文会把人工智能同物联网(IoT)融合起来，创建一个闭环运转的智能督导系统^[1]。图像识别来判定投放行为，积分制对正确的行为给予即时的正向回馈，设备端利用物联网状况管理并发送主动报警信息^{[2][3]}。

1.2 国内外研究现状

1.2.1 国外研究现状

国外垃圾分类起步早，更多是重“硬件”重“标准”。日本依靠社区细致分类模式加长期居民教育，居民参与度很高，德国经由生产者责任制延伸并加“绿点”体系，包装废弃物回收率长久处于世界前列。但是这些模式成本高或者规格依赖性强，不能直接应用于我国社区人口密集、垃圾种类繁杂的环境中^[4]。

1.2.2 国内研究现状

国内起步比较晚，但是在强有力的政策推动下发展迅速。高校和企业对于算法、终端、平台等做了大量的尝试并且取得了一定的成果，但是普遍存在的问题是“点上很强大，链路不顺畅”，有些仅仅追求识别准确率，忽略和清运流程的对接，有些只是发出满溢警报，没有形成对居民行为的激励与引导^[5]。单一物质激励会很快产生效果，但如果没有长期机制，习惯就会被反弹^[6]。破解以上问题要依靠一个完整的数据及责任链条来运行起来，即可以追溯、可以激励、可以闭环^[7]。

1.2.3 现有研究空白总结

通过对国内外垃圾分类相关研究以及应用情况的综合比较，发现目前智能垃圾分类督导领域还存在着核心的研究空白，这也是本文需要突破的地方。

AI 对垃圾分类复杂场景的适应性没有做个性化的设计。现有的模型大多采用通用目标检测算法，对于社区环境中存在的遮挡、堆叠、光照变化、垃圾形态不规则等问题没有很好地解决，同时又存在实时性和轻量化的部署问题^{[8][9]}。

缺少对积分激励机制进行精细化设计以及长期效果的检验。大多数系统只使用简单的积分累计方式，并没有结合投放频次、准确率、时段等因素来差异化激励，容易造成居民参与热情的降低，缺少可持续行为引导的能力^{[10][6]}。

多端协同全流程督导闭环没有真正打通。目前的平台大多只是实现了某项功能，并没有把居民投放、AI 识别、积分激励、设备告警、清运调度、后台监管形成一个完整的数据链条，造成信息孤岛和业务断层^{[11][12]}。

1.3 研究内容与目标

围绕把链路打通，本课题以搭好多端协同的架构为研究重点。识别要做快做准。社区场景要求实时性强并且成本可控，在精度满足的基础上选择 YOLOv8s 与领域迁移联合改进的方法，在上屏即时得到反馈的同时达到经济实惠的目的^[13]。

其次创建起可持续的积分激励机制。构建以投放、核验、记录、兑换为闭环的流程，将正确的投放转变为可以被量化的收益，使动力可见、可持续。

最终将设备运维信息化。对覆盖设备激活、心跳保持和异常告警等实施管理，并且设备状态可以在线查看，满溢或者离线则会发出预警并报警^[12]。

1.3.1 论文核心创新点

创建边缘云协同的垃圾分类轻量化识别结构。根据 YOLOv8s 的优化来实现社区垃圾场景的实时识别，使用边缘端实时识别、云端数据更新的方式，以保持较高的识别精度为前提降低硬件成本，提高复杂投放场景下对投放者的适应能力。

提出分层积分激励机制，做到精细。设计基础积分、准确率奖励、连续投放激励，用兑换体系和行为数据画像，达到对居民分类投放行为的长效、智能引导，解决传统的激励方式效果短暂问题^{[10][6]}。

以物联网为驱动设备实现数字化运维。建立设备激活、心跳保持、出现异常时自动上报、满溢告警联动机制，把原来的“手工巡检”变成数字化的“自动运维”，提高小区垃圾分类设备管理效率和系统稳定度。

1.4 论文组织结构

全文共分为七个章节，第一章为系统设计背景与价值，在国内外研究脉络下提出的问题，在第二章进行相关技术综述，第三章给出需求、功能与非功能同时说明，第四章给出系统设计，包括架构、数据及关键流程，第五章具体实现部分，第六章测试与评估部分，第七章总结并提出改进方向。

第二章 相关技术综述

2.1 前端及移动端开发技术

2.1.1 前端

系统前端如图 1 所示，使用 Vue.js 框架，主要是对智能垃圾分类督导系统数据看板场景的实现，Vue 的双向数据绑定特性可以将垃圾分类数据、设备状态、积分统计等核心信息实时更新^[21]。

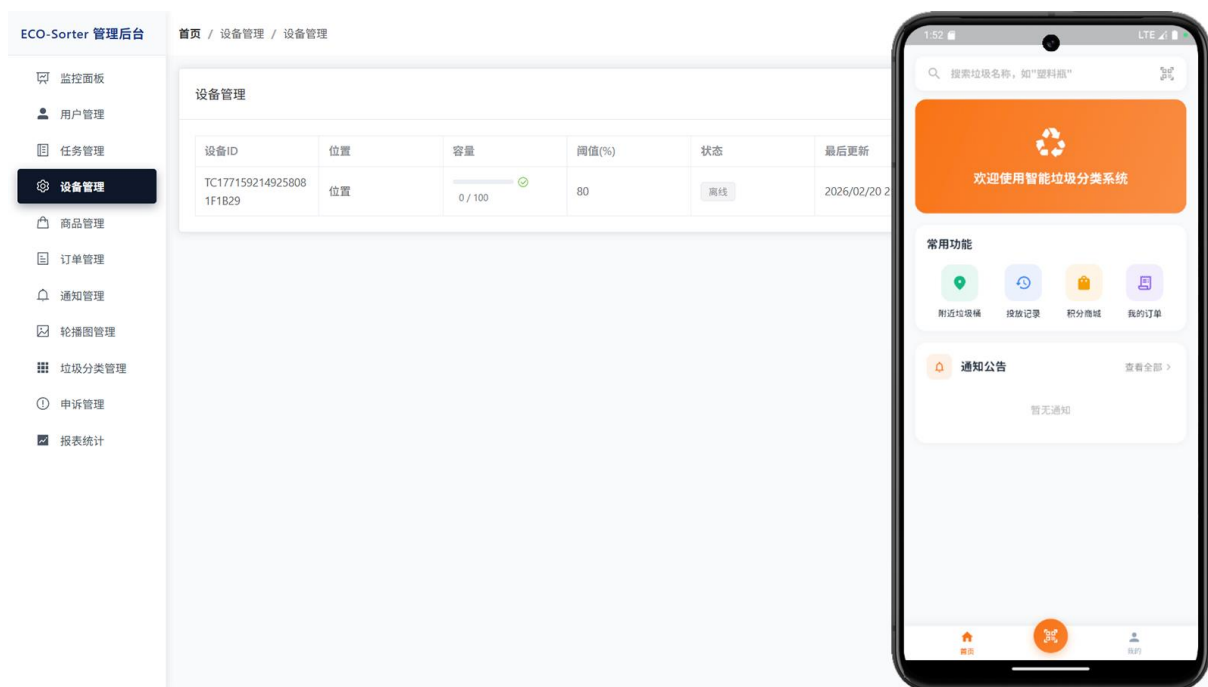


图 2.1 前端界面展示图

组合式 API 把设备告警、积分核算、用户行为分析这些模块拆散成可以互相独立运行的子程序，很好地符合了系统的多角色（居民/收集员/管理员）功能划分需求。

2.1.2 移动端

本系统移动端采用 Flutter 进行开发，适用社区居民使用的所有终端，包括安卓、苹果等各系统，Flutter 的跨端一致性可以保证在不同的系统中使用相同的一套代码，减少开发成本。

2.2 后端及数据库技术

2.2.1 后端框架

选择 Spring Boot 3.1，智能垃圾分类督导系统需要快速落地并且适应社区多设备、多场景的部署要求，Spring Boot 少配置、快速搭建生产环境的特点，可以缩短系统从开发到社区试点部署的时间周期，满足基层场景下快速验证、快速迭代的要求^{[14][15]}。

MyBatis Plus 对系统中居民投放记录、积分流水、设备状态、告警信息等大量的结构化数据增删改查操作进行适配，MyBatis Plus 的标准 CRUD 接口可以减少样板代码，提高数据操作层的开发效率。

2.2.2 安全鉴权

系统中垃圾投放终端长期在线，JWT 无状态特性不需要服务端保存会话，可以防止大量的设备连接造成内存压力，有利于设备在跨网关、跨节点之间访问后端服务，适应物联网分布式部署的环境^[16]。

2.2.3 数据库

使用 MariaDB 而不是 MySQL 是由于系统是面向国内社区场景，含有大量的中文数据（居民姓名、地址、垃圾品类名称、告警描述等），MariaDB 默认采用 utf8mb3 客户端字符集，与 MySQL 默认的 latin1 字符集相比，可以直接解决中文乱码问题，不需要再做字符集适配配置。

2.3 AI 识别与物联网技术

2.3.1 AI 识别

为满足社区垃圾分类督导场景下对于识别准确率、推理速度以及设备部署成本三个要求，选择使用了 Yolo8s 这个主要的识别模型，同时对它的几个版本进行了性能上的比较研究，如表 1 所示^{[13][9]}。

表 1 YOLOv8 不同版本模型性能对比

模型版本	大小 (MB)	推理速度 (ms)	精度 (mAP50 - 95)	适配性说明
YOLOv8n	12.8	28.0	37.3	精度过低，易误判垃圾品类，不符合分类督导需求
YOLOv8s	28.8	45.0	44.9	平衡社区低配智能设备的部署成本（体积仅 28.8MB）、实时性（推理 45ms 可实现“上屏即反馈”）与识别精度，适配社区场景核心需求
YOLOv8m	47.0	70.0	50.2	体积大、推理慢，部署需更高配置硬件，抬高社区设备采购成本，性价比低

将 AI 识别服务从 Java 主业务中分离为 Python 多服务。垃圾分类识别是计算密集型任务，在 Python 生态下深度学习模型的部署和更新更加容易，可以降低模型升级新垃圾品类等成本^{[17][18]}。

Java 主要用来完成积分核算、设备状态管理、用户权限等核心业务逻辑，两者各自承担着不同的职责，不会让 Java 加深集成深度学习框架造成耦合度高、运维繁杂的局面，符合

系统“识别 - 业务 - 激励”闭环运转的需求^{[7][13]}。

2.3.2 物联网技术

（1）设备状态管理

心跳、状态上报分系统方案。社区垃圾投放设备多，分散且数量大，固定间隔轻量心跳可以低成本保持设备在线状态监测，防止全量状态上报造成网络带宽的浪费；^{[19][20]}

只有当垃圾满溢、设备出现故障等重要情况发生的时候才向核心系统进行通报，从而满足在线可见的基本要求，同时考虑到一些老旧社区带宽比较小的情况^[20]。

（2）Nginx + Docker/K8s

Nginx 作为一个统一网关来屏蔽后端服务的细节，可以将居民端、设备端、管理端的请求分配到不同的服务上，在社区高峰期（早 / 晚投放时段）的高并发情况下也能保证系统的稳定运行，静态资源缓存可以减少回源的压力，提高垃圾分类识别结果和积分信息的显示速度^[7]。

第三章 系统需求分析

3.1 系统总体概述

本章从实际使用的场景入手，以居民一次投放为线索，从上到下拆解出角色能力及系统的需求。降低耦合，把系统拆分成四个互相独立的终端，即居民侧移动 App、收集员作业端、管理后台、投放点的智能垃圾桶设备端^[21]。

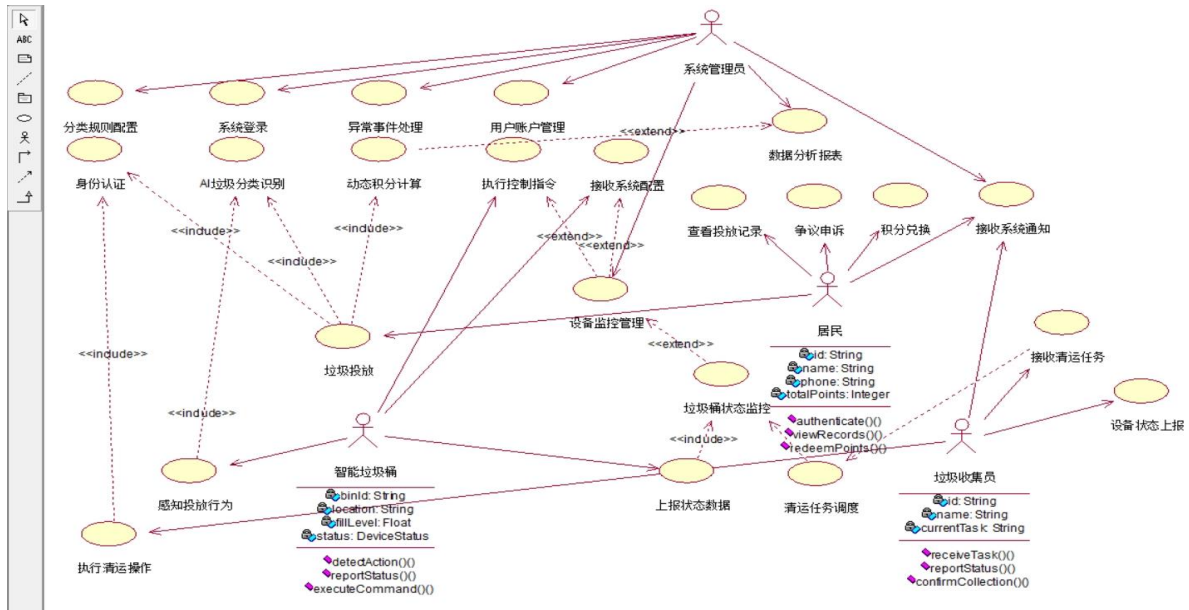


图 3.1 用例图

不同角色的重视程度也不一样，居民最关注投放是否正确、能否得积分，收集员最关心的是桶满度和路线安排，管理侧最关心的是规则、宣传和投诉治理，设备端最看重的是连接稳定性。因此把流程串起来形成识别、记录、复核、积分、调度、处理和反馈的闭环结构。

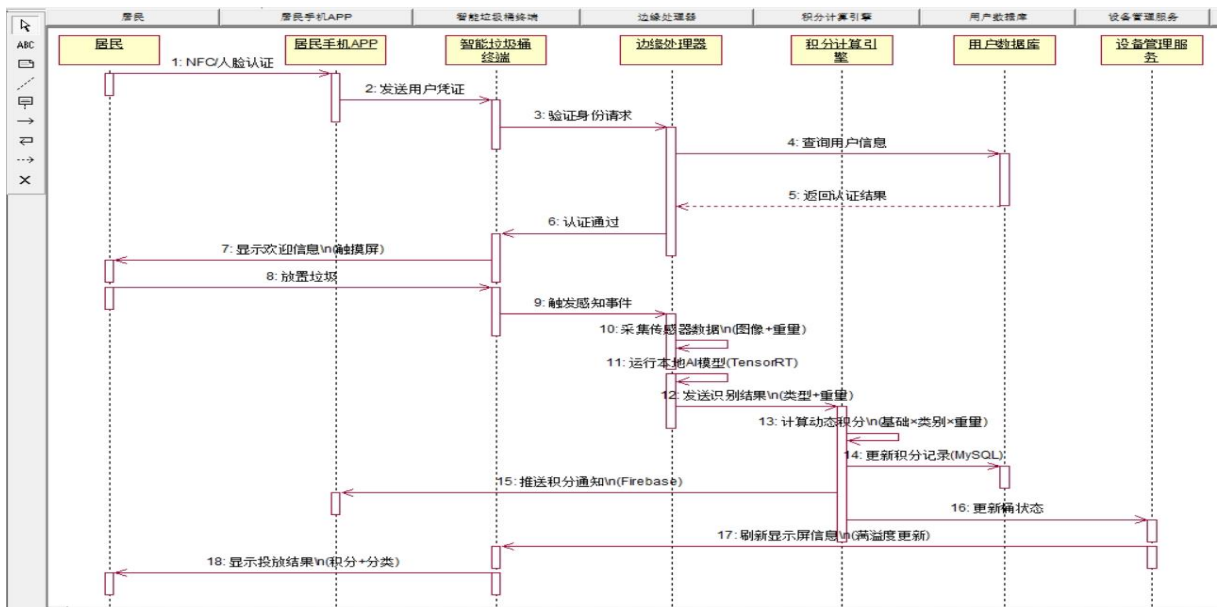


图 3.2 时序图

以投放为例，在设备端扫脸（或者游客）进流程中，摄像头抓拍上传到识别服务上，得到类别和置信度后，服务端会把通用标签映射到四分类上，根据这个来进行计分，之后还会产生分类记录并且关联到用户以及设备上，容量达到阈值就会触发告警并产生清运任务，收集员按照任务出发处理，并留存影像，管理后台看板可以随时刷新主要指标，异常则会经过工单来跟进直到关闭为止。

3.2 功能需求分析

3.2.1 居民用户功能需求

居民侧存在的主要问题有分类不清和缺少动力两个方面。因此居民端从“看得懂、反馈快”入手，识别为入口、积分为强化机制，两者在单次投放中形成闭环来提高参与度^{[10][6]}。

认证使用扫码、扫脸等方法，都坚持最小采集原则，人脸生物特征用 128 维向量加密保存，不保存人脸图像文件^[22]。识别结果实时回显、置信度。

3.2.2 收集员功能需求

收集员端的目标就是效率高、过程稳定、少空跑。为了达到精准调度的目的，在端上就要及时了解容量及在线状态，当容量大于阈值（例如 80%）或者长时间不在线的时候，系统就会发出警告，并且给出优先处理的建议。

核心能力来源于容量阈值自动触发的自定义任务、定时触发的例行任务和手动分配的任务一起被纳入同一个视图进行管理。

3.2.3 管理后台功能需求

管理后台是“整体的视角、治理的效率”。看板要展示投放高峰、参与度、积分发放这些重要指标，为调度和策略调整给予支撑^[11]。日常运维包含用户权限、设备、积分商城、订单、工单等模块，投诉要形成可以追踪、可以闭环的处理路径。

看板主要展示在线率、识别成功率为 95%、设备容量告警、居民参与度、积分发放、履约成本等；权限按照组织/区域、角色进行分级授权以防止越权；设备管理包含台账、激活、心跳、状态监测等功能并且可以远程下发参数、升级、重启；调度和工单都是根据告警自行触发，可以二次指派、超时升级、闭环评价。

3.2.4 智能垃圾桶端功能需求

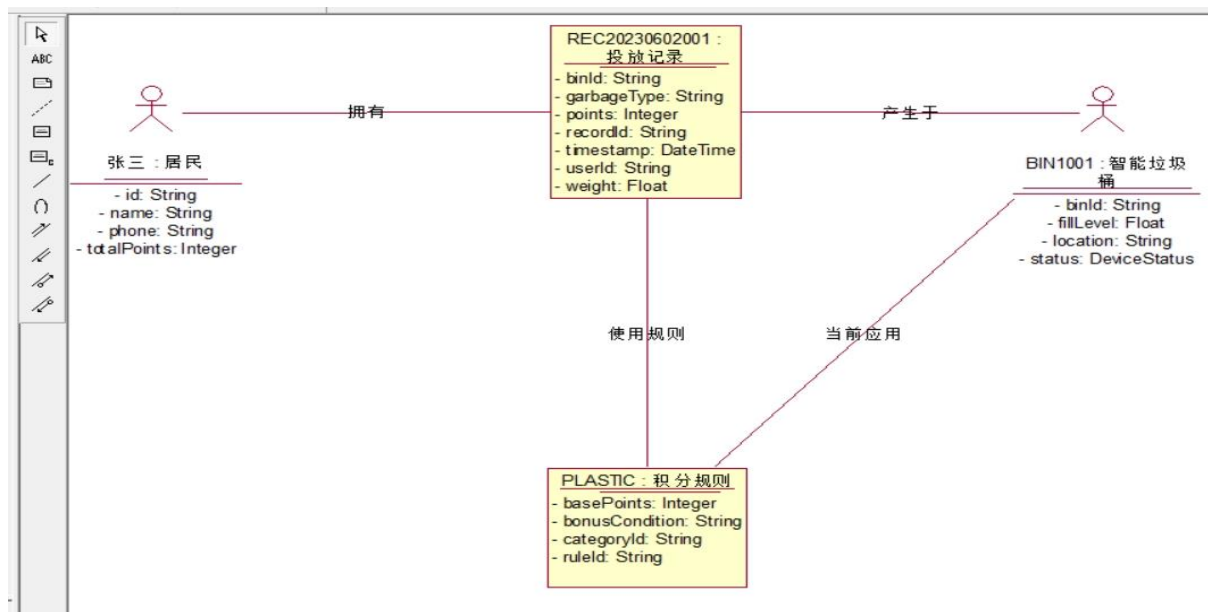


图 3.3 对象图

设备端处于投放一线，集数据采集和人机交互于一体。新设备要在管理员注册成功之后才能使用。采用扫码登录的方式，不用在终端上手动输入账号密码。摄像头抓取、上传识别、目标在 3 秒内给出类别和置信度，保证在线，每 30 秒发心跳^[19]。

3.3 非功能需求分析

可用性上，识别和记分是用户最直接的体验点，不能有卡顿。月可用率 $\geq 85\%$ ，核心接口登陆、识别入库、任务分配可用率 $\geq 80\%$ 。

原型联调结果显示用户在等待时间大于 3 秒的时候很容易感觉到出现了问题，所以端到端的延迟(P95 小于等于 3 秒，P99 小于等于 5 秒)作为最终要求被提出，后台高频查询(P95 小于等于 200 毫秒)是被考虑进去的要求，服务器需要有扩展性的能力来应对高峰期。

遵循《个人信息保护法》采集前告知、最小必要、敏感数据分级分类加密存储、通信全过程使用 TLS、权限最小化以 RBAC 落地最小授权、关键操作和数据访问需要审计留痕^[10]。具有生物特征的链路使用独立密钥管理系统及访问审查制度加以管理。

3.4 系统可行性研究

技术可行性方面后端用 SpringBoot，前端用 Vue.js，识别使用 YOLOv8s 在速度和精度之间取得平衡，在资源受限的设备上也进行了原型验证。

经济可行性，模型量大但推理成本可控，采用容器化和弹性伸缩方式，资源消耗处在可以承受的范围之内，积分商城依靠合作和定额控制来管理成本，费用构成可以预测。

操作可行性在于流程符合居民投放、收集员的作业习惯，学习成本低；管理后台有报表、工单功能来支持日常治理。

合规性可行，敏感数据依照分类分级管理，出入库要有审批并且留下全部痕迹，隐私政策对外发布并提供撤回授权途径。

社会可行性指的是系统的设计符合居民的日常投放习惯，简化分类的操作步骤，并加上积分奖励措施等，可以有效地提高居民参与的积极性，得到社区管理者以及居民的认同和支持，为系统在社区的推广提供保障。

第四章 系统设计

4.1 系统架构设计

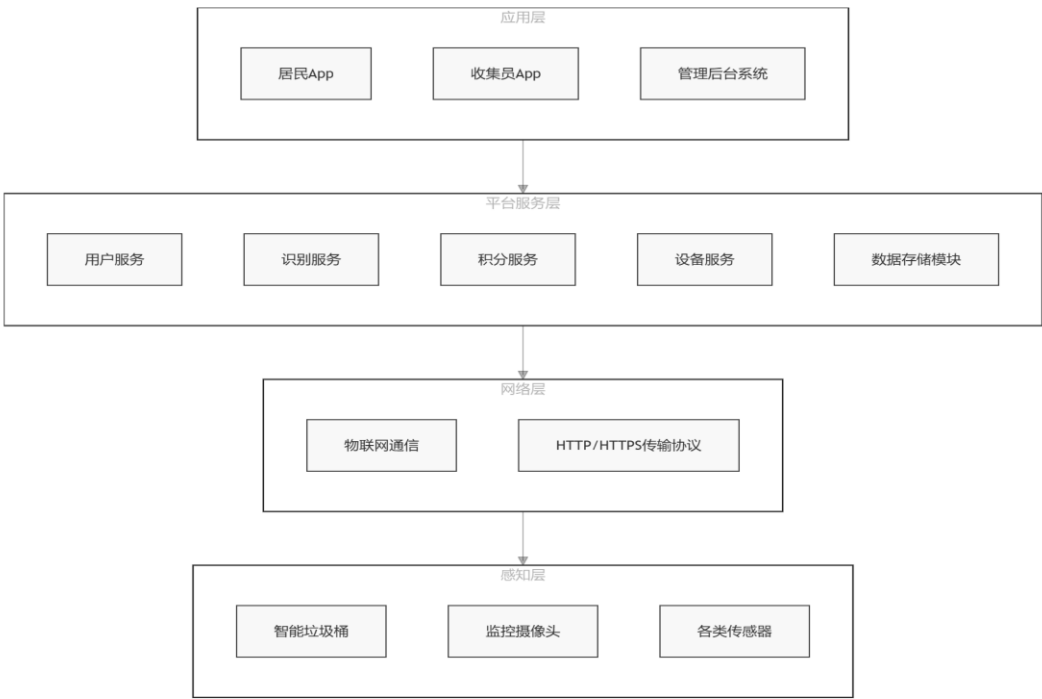


图 4.1 系统架构图

在整理出业务场景和限制之后，就选择了前后端分离的多服务分布式结构^{[1][15][23]}。在此模式之下前后端可以分别进行开发、部署以及升级工作，从而大幅削减联调费用以及发布之间的联系。

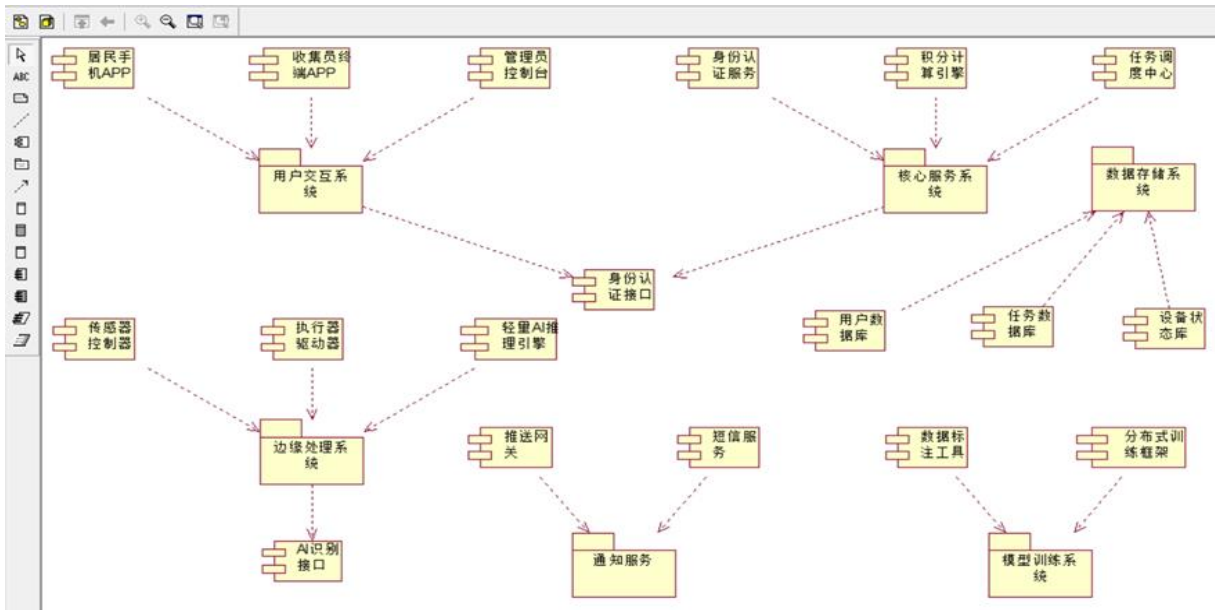


图 4.2 系统组件图

平台层做核心业务、数据存储和任务调度。系统采用多服务分布式架构，把用户、积分、设备、订单等模块拆分成独立的服务，服务之间使用 REST 通信，可以按负载独立扩展。推理耗时和计算量很大，于是将 AI 推理从业务端独立出来作为 Python 服务运行，业务服务主要做流程和规则处理，AI 服务专门做模型以及性能优化。两者各司其职，扩容和版本升级互相独立。

应用层面向最终用户，根据角色分别给居民端 App、收集员端和管理后台。居民端重在引导正确投放、及时反馈识别和积分，提供便捷的兑换方式，收集员端重在任务分配、路线建议、作业指导，管理后台重在监控、分析、运维。

4.2 核心机制设计

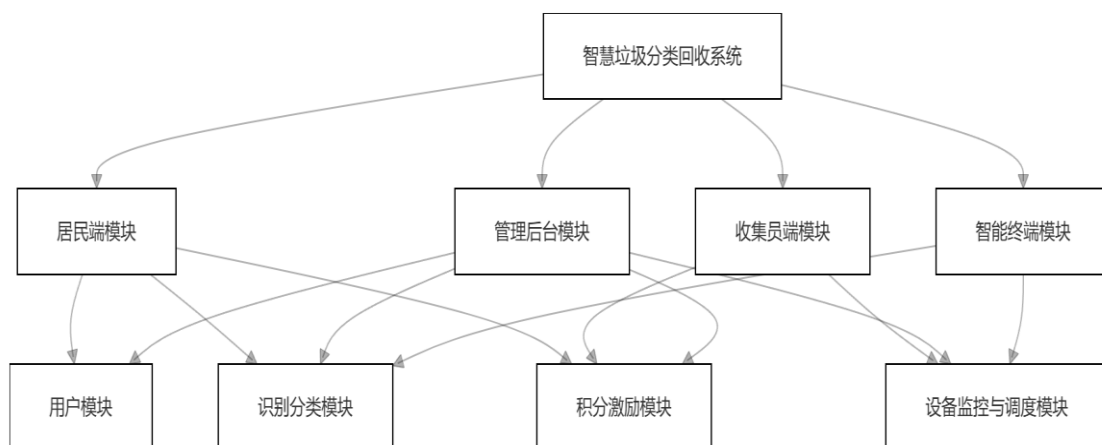


图 4.3 系统功能模块图

4.2.1 安全认证与鉴权机制

多终端接入、弱网环境并存，对认证的要求就是稳定可靠。传统的 Session 是依靠服务端会话状态，当断线重连或者实例扩容的时候会造成同步压力，所以选择了无状态的 JWT 方案^[20]。此方案减小了服务对于会话存储的依赖，利于横向扩展。

鉴权上，用户登陆或者设备激活之后，服务器会发给用户一个包含用户 ID、角色以及过期时间的 Token，客户端把 Token 本地存储起来，在以后的请求中以 Authorization 的形式添加到头部中去。服务端只对签名、有效期进行校验，不查询会话状态。这样既简化了实现，又可以自然地扩容。

4.2.2 基于 YOLOv8 的垃圾识别逻辑

模型选型对比 Faster R-CNN、SSD 与 YOLO 系列后，综合速度与精度选择 YOLOv8 作为识别模型^{[13][9]}。设备抓拍之后上传到 AI 服务，用加载好的模型做推理，得到类别和置信度

[24]。

模型侧给出的大部分为通用物体标签，如塑料瓶、纸箱等，业务层需要将其映射成四类。映射表可以实现一对多的配置，积分策略会根据置信度的变化而做出调整，高置信度时用常规方式入账，置信度较低时减免或者不记分，在激励和误发控制之间做出权衡。

在图像方面按照长边进行缩放并且使用 letterbox 形成统一的大小，骨干网络及 FPN 抽取不同尺度的特征，解耦出的头分别给出边界框和类别评分，之后用阈值过滤加上 NMS 去重，得到 Top-k 结果。业务映射、积分策略和识别结果三者保持一致，把识别、计分和提示这条链路连接起来。

```
COCO
COO_TC_GARBAGE = {# 生活用品/可回收24: {
24:  name": "backpack": "category": "可回收物", "cn名称": "背包"}
25:  name": "beile", "umborlea": "可回收物": "cn名称": "伞"}
26:  name": "hancr", "handbag": "可回收物"}
27:  name": "tie", "cuttegrory", "cn名称": "手提包"}
28:  name": "tie", "suitecase": "可回收物", "cn名称": "行李箱"}
29:  # 运动用品((可回收))
29:  # #运动用品(可回收)29: { "frisbe": "可回收物"; "cn名称": "飞盘"}
30:  name": "skis", "wine glass": "可回收物", "cn名称": "滑杯板"}
31:  name { : "sAoknon", "category": "滑雪单板" )
32:  name": "basebell bat", "n名称": "可回收物", : "其他"风箏"}
34:  name": "basebell glove", : "可回收物": "棒球手棒"}
35:  name": "skatboard", : "可回收物"}
36:  name": "运球手套"}
31:  name": "baskell glov", "cn名称": "滑板"}
37:  name": "surfboard", : "可回收物"}
42:  cu具 knife "杯子)
48:  na3: { : "fork": "tennctet", : "可回收物"}
45:  bowl": "网球拍")
39:  # #瓶罐/餐具(核心垃圾)39: { "name": "bottle", "speont": "橙子"}
46:  # 食罐/餐具(核心垃圾)香蕉)}
47:  apple": "sandwiiicor": "苹果明治}
48:  name": "orange_ball": "cn名称": "橙子}
50:  name_brococily", "category/厨回垃圾", : "西兰花}
```

图 4.4 物品类别映射

4.2.3 智能垃圾桶端扫码登录机制

扫码登录是设备、手机、服务器三方互相配合完成的，采用状态机驱动使流程的安全可控。设计初衷就是解决户外终端不方便输入账号密码的问题，只需要使用 App 扫码屏幕二维码就可完成登录，操作简单，提示清楚。

流程为设备申请临时会话，并以二维码形式展示出来，状态是“待扫码”，扫码后 App 解析会话并上报用户身份给服务器进行校验，之后将结果置为“已扫描”。设备短轮询，等待用户在手机端确认，确认之后生成正式的 Token，设备轮询到“已确认”之后获取并保存起来，之后在请求中携带这个 Token。整个过程不从设备端获取个人信息，风险面更小。

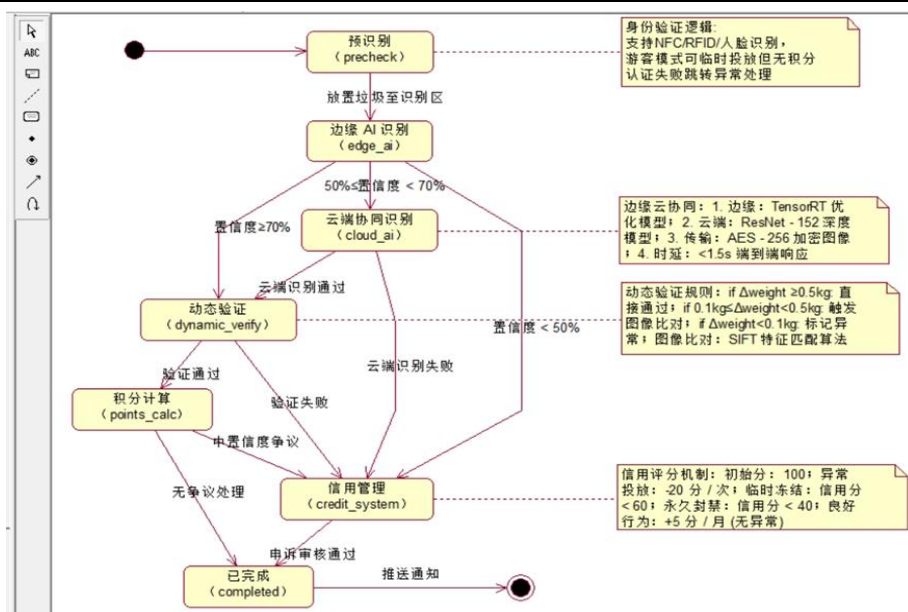


图 4.5 活动图

4.2.4 设备激活机制

新设备上线之前要先进行登记和激活，保证接入合法。后台登记元数据（型号、厂家、点位、区域、负责人等）之后，系统就会生成一个形式为“TC 加上时间戳再加上 UUID”的唯一的编号。编号既可以保证全局唯一，又便于从编码中看出登记时间和设备来源。

首次启动时设备用编号和硬件标识发起认证；服务器核对登记和状态之后签发一个有效期为一年的 token 并返回。设备持久化保存 Token，在即将过期的时候主动续期，防止频繁认证造成的链路中断。该机制可以不牺牲安全来减少认证开销。

4.2.5 设备心跳机制

设备处于户外弱网，需要通过心跳来判断在线状态。心跳间隔设置为 30 秒，保证及时性的同时也考虑到带宽的占用情况，服务端记录每个客户端的最后一次心跳时间，并且用连续 3 次无回应来判定该客户端是否离线以及发出告警。

心跳只含设备 ID、时间戳和签名，不带业务字段，减少丢失以及带宽占用。容量、温度等状态采用独立接口进行逐个上报，防止出现心跳膨胀的情况发生；当出现异常的时候，则发出告警信号，并且产生任务，从而借助事件来驱动不必要的重复上报过程的出现。

4.3 数据库设计

4.3.1 概念模型设计

数据库概念模型以用户、设备、分类记录这三个实体为中心展开，用户实体包括账号、积分、人脸特征等，设备实体包含编号、点位、状态、心跳等，分类记录起到连接作用，保存了谁在哪儿台设备上投放了什么、识别的结果、图像的索引、积分的变化等内容。

为了保证完整的业务流程闭环设计了积分商品、兑换订单、回收预约、投诉工单、清运任务等辅助实体，并且增加了积分规则、设备告警、系统日志、操作审计这四种扩展的实体，

分别用来承担分层积分策略配置、设备异常全过程追溯、系统运行记录和重要操作审计留痕的功能。

4.3.2 逻辑设计与优化

逻辑层一共设计了 18 张表，按照第三范式来控制冗余并减少更新异常。体量最大的表是分类记录表，在使用过程中迅速增加，规模可以达到百万至千万。为加速检索，对 user_id、device id、create time 等高频条件建立索引，覆盖按用户与设备的常见查询。

存储引擎使用的是 InnoDB，用事务、行锁和外键来实现。积分扣减和订单生成都以原子的方式被包含到一个事务当中，任何一个出现失败都会引起整个事务的回滚，保证账务的一致性。

用户表保留 face_encoding 字段，给之后人脸登录扩展留出位置，防止后期对表结构进行更改。

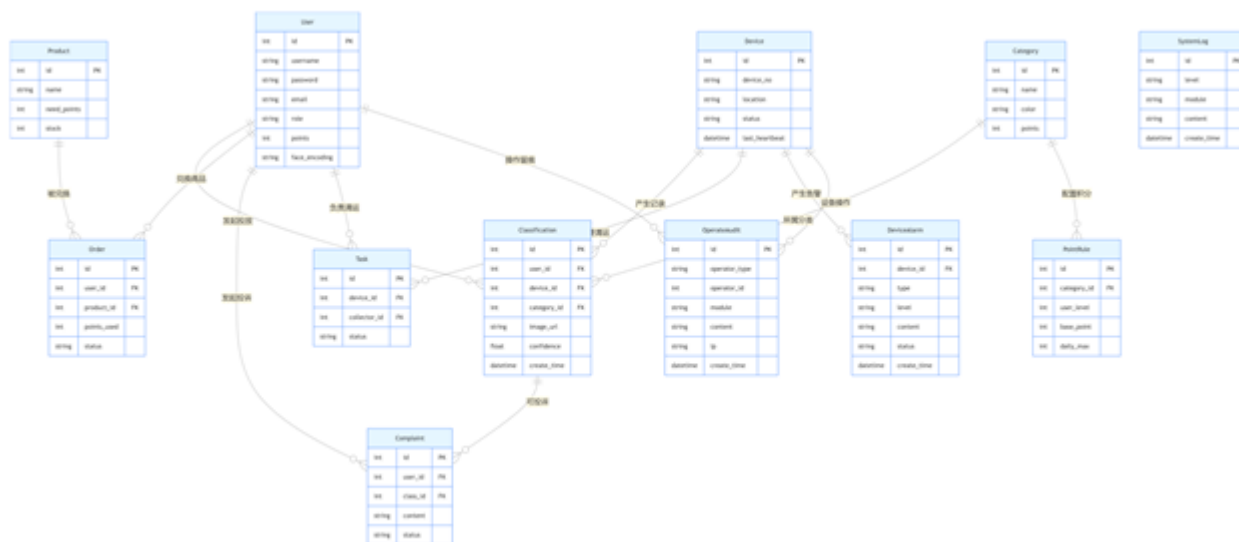


图 4.6 数据库 ER 图

4.3.3 索引策略设计

分类记录为体量最大的表，随着系统的使用数据量不断增大，索引的设计以高频查询场景为主，具体的参考表 2，未来考虑分库分表。

表 2 索引策略设计

表名	索引类型	索引字段	设计目的
classifications	组合索引	user_id+created_at	“按用户查询历史投放记录”“统计用户投放频次” 等高频场景
classifications	单字段索引	waste_category_id	“按垃圾品类统计投放量” 的业务分析查询

表名	索引类型	索引字段	设计目的
classifications	单字段索引	trashcan_id	快速定位某设备的所有投放记录, 支撑设备投放量统计
users	唯一索引	username/email	保障账号唯一性, 加速用户登录、注册校验
trashcan_data	单字段索引	device_id/status	查询设备状态、定位离线 / 满溢设备
point_records	组合索引	user_id+created_at	支撑“用户积分流水查询”“月度积分统计”场景
collection_tasks	组合索引	collector_id+status	筛选收集员待处理 / 已完成任务
qr_login_sessions	单字段索引	status/expires_at	扫码登录状态校验, 支撑定时清理过期会话
products	单字段索引	status/category	商品上架状态查询、商品分类筛选
orders	单字段索引	user_id/status	查询用户订单、按订单状态筛选
bookings	单字段索引	user_id/status/appointment_date	查询用户预约记录、按时间与状态筛选
complaints	单字段索引	user_id/status/created_at	查询用户申诉、按处理状态筛选
point_rule	单字段索引	waste_category_id/user_level/status	快速匹配分层积分规则
system_log	单字段索引	created_at/module	按时间、模块快速检索运行日志

4.3.4 核心表设计

表 3 用户表设计

字段名	数据类型	主键	外键	约束	说明
id	BIGINT	是	否	AUTO_INCREMENT, NOT NULL	用户 ID, 主键
username	VARCHAR(20)	否	否	UNIQUE, NOT NULL	用户名, 唯一
email	VARCHAR(100)	否	否	UNIQUE, NOT NULL	邮箱, 唯一
password	VARCHAR(255)	否	否	NOT NULL	密码
role	VARCHAR(20)	否	否	NOT NULL, DEFAULT 'RESIDENT'	角色: 居民 / 收集员 / 管理员 / 垃圾桶设备
is_active	BOOLEAN	否	否	NOT NULL, DEFAULT TRUE	是否激活

字段名	数据类型	主键	外键	约束	说明
avatar_url	VARCHAR(500)	否	否	NULL	头像 URL
phone	VARCHAR(20)	否	否	NULL	联系电话
address	TEXT	否	否	NULL	地址
current_points	INT	否	否	DEFAULT 0	当前积分
face_encoding	TEXT	否	否	NULL	人脸特征 JSON
face_verified	BOOLEAN	否	否	DEFAULT FALSE	是否已人脸验证
last_login	DATETIME	否	否	NULL	最后登录时间
created_at	DATETIME	否	否	NOT NULL, DEFAULT CURRENT_TIMESTAMP	创建时间
updated_at	DATETIME	否	否	NOT NULL, DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	更新时间

表 4 积分规则表设计

字段名	数据类型	主键	外键	约束	说明
id	BIGINT	是	否	AUTO_INCREMENT, NOT NULL	规则 ID, 主键
waste_category_id	BIGINT	否	是	NOT NULL	关联垃圾类别 ID
base_point	INT	否	否	NOT NULL	单次投放基础积分
min_confidence	DECIMAL(5,4)	否	否	DEFAULT 0.5	最低有效识别置信度
daily_max_times	INT	否	否	DEFAULT 10	每日最多有效投放次数
daily_max_point	INT	否	否	DEFAULT 100	单日积分上限
status	TINYINT	否	否	DEFAULT 1	状态: 1 - 启用 0 - 禁用
description	VARCHAR(255)	否	否	NULL	规则描述
created_at	DATETIME	否	否	NOT NULL, DEFAULT CURRENT_TIMESTAMP	创建时间

字段名	数据类型	主键	外键	约束	说明
updated_at	DATETIME	否	否	NOT NULL, DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	更新时间

表 5 投放记录表设计

字段名	数据类型	主键	外键	约束	说明
id	BIGINT	是	否	AUTO_INCREMENT, NOT NULL	分类记录 ID
user_id	BIGINT	否	是, 关联 users.id	NOT NULL	投放用户 ID
trashcan_id	BIGINT	否	是, 关联 trashcan_data.id	NULL	投放垃圾 桶 ID
waste_category_id	BIGINT	否	是, 关联 waste_categorie s.id	NULL	识别的垃 圾类别 ID
image_url	VARCHAR(500)	否	否	NULL	分类图片 URL
confidence_score	DECIMAL(5,4)	否	否	NULL	AI 识别置 信度
ai_suggestion	VARCHAR(100)	否	否	NULL	AI 识别建 议
user_feedback	VARCHAR(20)	否	否	NULL	用户反馈
corrected_category_id	BIGINT	否	是, 关联 waste_categorie s.id	NULL	用户更正 类别 ID
notes	TEXT	否	否	NULL	备注
ip_address	VARCHAR(45)	否	否	NULL	操作 IP
user_agent	VARCHAR(500)	否	否	NULL	客户端信 息
created_at	DATETIME	否	否	NOT NULL, DEFAULT CURRENT_TIMESTAMP	投放时间

字段名	数据类型	主键	外键	约束	说明
updated_at	DATETIME	否	否	NOT NULL, DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	更新时间

表 6 垃圾品类字典表设计

字段名	数据类型	主键	外键	约束	说明
id	BIGINT	是	否	AUTO_INCREMENT, NOT NULL	类别 ID
name	VARCHAR(50)	否	否	UNIQUE, NOT NULL	类别名称: 可回收 物 / 有害垃圾 / 厨余垃圾 / 其他 垃圾
name_en	VARCHAR(50)	否	否	NULL	英文名称
description	TEXT	否	否	NULL	类别描述
color	VARCHAR(7)	否	否	NULL	颜色代码
icon	VARCHAR(50)	否	否	NULL	图标
disposal_instructions	TEXT	否	否	NULL	投放说明
disposal_method	VARCHAR(200)	否	否	NULL	处理方式
environmental_impact	TEXT	否	否	NULL	环境影响
recycling_rate	DECIMAL(5,2)	否	否	DEFAULT 0.00	回收率
common_items	TEXT	否	否	NULL	常见物品
special_handling	BOOLEAN	否	否	DEFAULT FALSE	是否特殊处理
hazardous	BOOLEAN	否	否	DEFAULT FALSE	是否有害垃圾
is_active	BOOLEAN	否	否	DEFAULT TRUE	是否启用
sort_order	INT	否	否	DEFAULT 0	排序
points	INT	否	否	DEFAULT 0	投放积分
created_at	DATETIME	否	否	NOT NULL, DEFAULT CURRENT_TIMESTAMP	创建时间

字段名	数据类型	主键	外键	约束	说明
updated_at	DATETIME	否	否	NOT NULL, DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	更新时间

4.3.5 二维码登录会话表设计

二维码登录采用独立的会话表来管理状态，会话 ID、设备 ID、用户 ID、状态、创建时间和过期时间等都会被存入会话表中。状态划分有“待扫码”“已扫描”“已确认”“已过期”等等，保证状态机流转以及有效期控制。

为了加快轮询查询的速度，在 status 上创建索引；定时清理任务来删除过期或者已经确认但是时间很久的记录，防止表变大，保证查询性能。

4.3.6 业务事务一致性设计

对于垃圾投放积分、积分兑换、设备数据上报等重要流程都用到了数据库事务来保证数据一致，防止出现数据多算、少算、丢失或者重复的情况。

用户投放垃圾并获得积分时，会将新投放的记录、更新用户总积分、写入积分流水这三步合并到一个事务中去。只要其中一步出错，整个操作就会回滚，不会出现投放成功了但是没有加积分、同一次投放重复给积分的问题。

用户用积分换购商品的时候，积分扣减、商品库存减小、创建订单、记载积分消耗流水都会成为一体一起执行。如果用户的积分不够、商品缺货、中间出现异常等，则所有的操作都将被取消，不能造成扣了积分而没有产生订单、或者商品被超卖等情形的发生。

4.4 系统接口规范

4.4.1 接口设计

接口设计遵循 REST 风格：HTTP 动词语义化，资源路径使用复数名词（如/api/users、/api/devices），用途明确、便于记忆与维护。

响应统一为 JSON 三段式：code、message、data。code 表示状态（200 成功，400/401/403/500 表示不同错误），message 提供人类可读说明，data 承载业务数据。全局异常处理统一错误响应格式，避免堆栈信息外泄。

为了同时达到效率和安全的目的而使用 GZIP 和 SSL。识别接口极简，只把图片 URL 传给服务端，服务端返回识别结果和积分变动，复杂的逻辑放在后面使用，减轻了前端的负担。

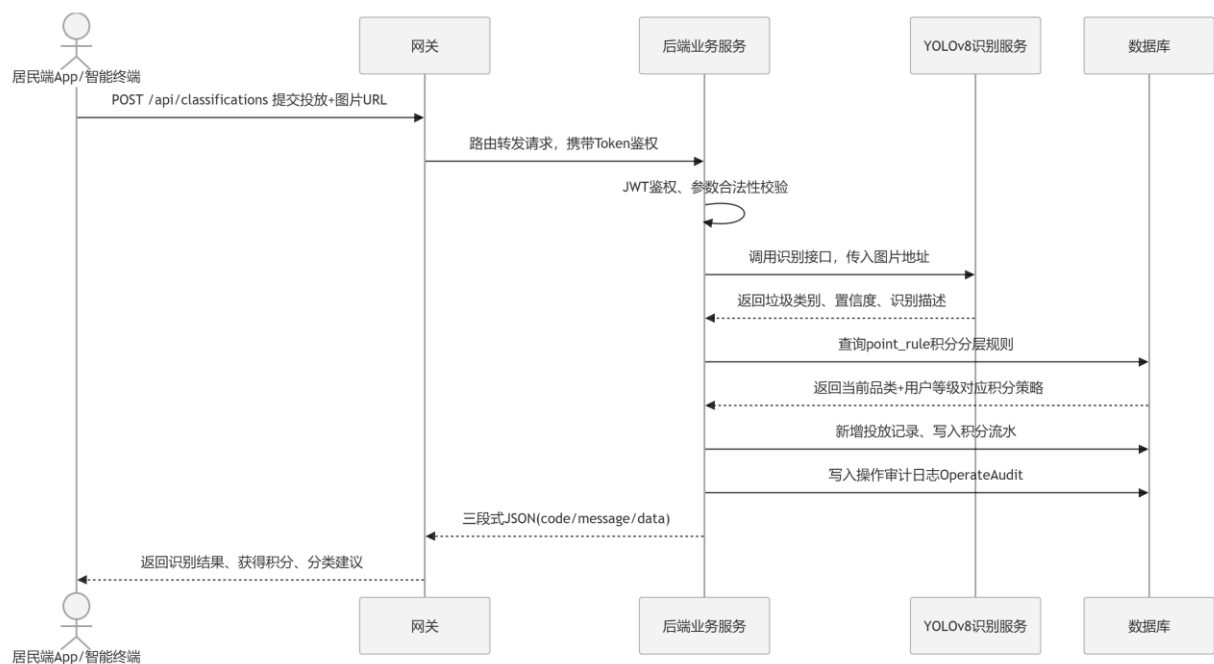


图 4.7 接口时序图

4.4.2 居民端模块设计

首页通知公告、积分变化、任务进度；分类拍照是主要的能力，图像上传、识别和引导，以及投放和积分的记录都在这里。

积分商城给用户 提供商品展示和兑换、订单详情及物流跟踪服务；预约回收主要针对大件及可回收物，可以选定回收时间，然后系统自动完成任务下达工作；个人中心包含资料更新、投放记录查看、积分统计以及反馈途径等部分内容，更加注重信息清楚而且方便。

4.4.3 收集员端模块设计

收集员模块以容量阈值、周期计划和人工指派三源合一的方式显示出来，到场后按照扫码开箱、清运、复位、影像凭证四个环节的流程完成循环，路线规划根据地理位置以及优先级给出建议路径，并且可以一键导航。

设备监控实时展 现在线状态、容量情况、温度状况等，可以较好地安排路线，个人统计按日/周/月形式体现完成数量、超时比例和投诉率，可以自我改进和考核。

4.4.4 管理后台模块设计

后台的目标就是“看得全、管得稳”。看板关注的是在线率、识别成功率、容量告警、参与度、积分发放、履约成本等指标，以实时化、可视化为主要特点。

设备管理涵盖台账、激活、心跳、状态、参数、升级、重启，覆盖全部生命周期，调度和工单按照告警自动创建任务，可以对二次分配任务进行分发，也可以设置超时自动升级为紧急任务，并且可以形成闭环的评价机制。积分和商城模块有规则设置、商品管理以及库存

管理、履约与对账的板块，在成本管控同体验改善两者之间寻找恰当平衡。

4.4.5 智能垃圾桶端模块设计

设备激活模块完成编号领取和 Token 签发,扫码登录模块用二维码来确定获得正式 Token 的方式,既方便又安全。状态上报模块定时上报容量、温度、故障码等信息,出现异常的时候发出报警信号。

4.4.6 业务流程设计

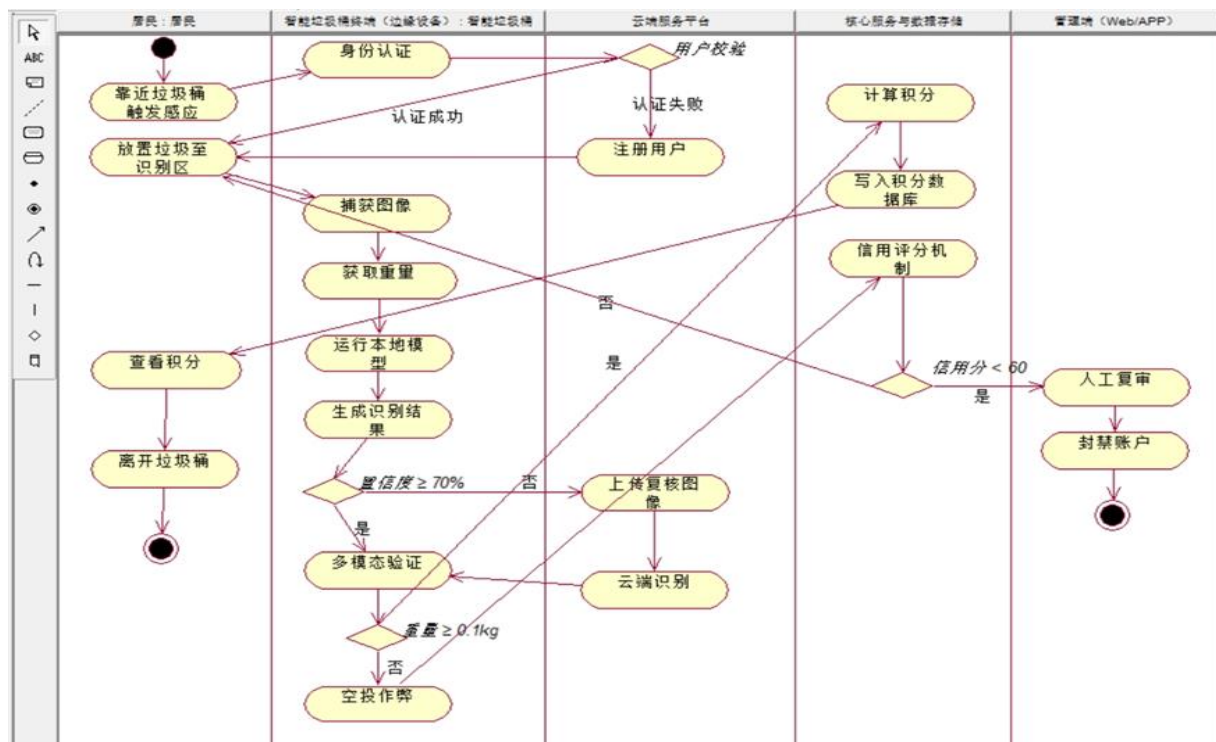


图 4.8 垃圾投放业务活动图

流程设计的主要目的就是跑通业务闭环,保证可追溯和异常闭环处理。覆盖注册登录、识别、积分兑换、回收预约、投诉、设备与任务管理等主要的场景。

注册登录作为入口,要平衡体验和安全,手机号加验证码注册并且做唯一性校验,成功之后签发 JWT,后续请求带上 JWT,登录支持短信、扫码、人脸识别等多种方式,可以根据习惯灵活选择,流程简单、上手快。

设备拍照上传, AI 服务用 YOLOv8 推理出类别和置信度,业务层对应法定的四个种类并打分,生成分类记录关联用户或者设备,并且更新积分,结果通过屏显和语音反馈的方式呈现出来,提高可理解性。

先校验余额再提交;余额充足即启动事务同时进行扣减和下单;订单产生之后给出详情以及物流追踪;核销码与快递两种履约对应不同的商品。

回收预约从用户发出到收集员处理、状态闭环更新的全过程打通,即用户选择物品和时间段,系统就地派单;接单之后和用户确认上门时间;完成之后更新状态,用户侧可以看到

进度。

用户填写信息并上传证据后，系统产生工单并派单到相应人员，处理流程分为受理、审核、处理等阶段，并且状态对用户是实时可见的，最终的结果会用来对流程进行改进。

登记生成编号、首次激活获得 Token、运行期心跳和状态监控、故障自动产生维修工单、生命周期终端执行报废，保证从入库到报废的一环不掉线。

容量大于一定值时就自动产生任务，并就近发送；作业按照标准程序进行并上传凭证；后台随时监督接单、开始、完成等各个环节的状态，提高调度的效率和透明度。

第五章 系统实现

5.1 后端实现

5.1.1 开发环境与项目结构

后端使用 Linux 服务器，框架使用 SpringBoot3.1，JDK 采用 LTS 的 Java17^{[1][9]}。依赖及构建工作交给 Maven 完成，数据库使用的是 MariaDB10.11。开发工具为 IntelliJ IDEA。

用分层架构把 Controller 用来接入、参数校验，把 Service 层用来做主要的业务逻辑实现，积分规则、设备激活、申诉审核等等都在 Service 层里面，getRepository 层采用 MyBatis Plus 进行数据交互，分层解耦可以大大提高代码的可维护性。

5.1.2 核心功能实现

5.1.2.1 用户认证实现

前端提交邮箱和密码，后端校验账号的合法性之后，使用 BCrypt 加盐哈希验证密码（同一个明文密码生成不同的哈希值），校验通过后使用 HS256 算法签发 JWT，Token 内包含用户 ID、角色以及过期时间（普通用户 24 小时，设备端 1 年）；^[16]

鉴权拦截 JwtAuthenticationFilter 拦截入站请求校验 token 的有效性，缺少或者过期返回 401 并触发前端登录跳转，新增接口防刷机制对同一个 IP 高频请求（1 分钟>20 次）做限流处理，防止恶意攻击。

```
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private static final String BEARER_PREFIX = "Bearer ";

    private final UserRepository userRepository;
    private final TrashcanDataRepository trashcanDataRepository;
    private final JwtUtil jwtUtil;

    public JwtAuthenticationFilter(UserRepository userRepository,
                                   TrashcanDataRepository trashcanDataRepository,
                                   JwtUtil jwtUtil) {
        this.userRepository = userRepository;
        this.trashcanDataRepository = trashcanDataRepository;
        this.jwtUtil = jwtUtil;
    }

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain filterChain)
        throws ServletException, IOException {

        String requestURI = request.getRequestURI();
        String authHeader = request.getHeader("Authorization");

        if (authHeader != null && authHeader.startsWith(BEARER_PREFIX)) {
            String token = authHeader.substring(BEARER_PREFIX.length());

            try {
                if (jwtUtil.validateToken(token)) {
                    String tokenType = jwtUtil.getTokenType(token);

                    if ("device".equals(tokenType)) {
```

图 5.1 用户登录认证后端代码

5.1.2.2 垃圾识别实现

云端 Python 服务部署, 接受边缘端上传的垃圾图像, 识别结果由边缘或者云端同步给 Java 后端, 后端按照品类基础分、置信度奖励分、频次激励分计算积分, 实现投放行为实时反馈 [17][13]。

```

async def recognize_garbage_with_file(file: UploadFile = File(...)):
    if cls_id in COCO_TO_GARBAGE and conf > best_confidence:
        best_confidence = conf
        best_detection = COCO_TO_GARBAGE[cls_id].copy()
        best_detection['confidence'] = conf

    if best_detection:
        category = best_detection['category']
        item_name = best_detection['cn_name']
        confidence = int(best_detection['confidence'] * 100)
    else:
        category = "其他垃圾"
        item_name = "未识别物品"
        confidence = 50

    category_id = get_category_id(category)
    advice = CATEGORY_ADVICE.get(category, '请正确分类投放')

    return RecognitionResponse(
        data=RecognitionResult(
            item=item_name,
            category=category,
            confidence=confidence,
            advice=advice,
            categoryId=category_id
        ),
        success=True
    )
except Exception as e:
    print(f"[ERROR] 识别失败: {str(e)}")
    raise HTTPException(status_code=500, detail=str(e))

```

图 5.2 python 图像识别模型推理代码

5.1.2.3 积分管理实现

积分采用“基础分 + 置信分”的策略。基础分根据类别回收价值来确定, 置信分和置信度相关联, 图像越清晰、特征越明显, 奖励越高。

PointService 封装 addPoints 与 deductPoints, 所有变动都会在 point_records 留痕, 便于追溯与申诉。

```

public class PointService {

    @Transactional
    public PointRecordResponse deductPoints(Long userId, Integer points, String type, Long referenceId, String description) {
        User user = userRepository.selectById(userId);
        if (user == null) {
            throw new ResourceNotFoundException("用户不存在");
        }

        Integer currentPoints = user.getCurrentPoints();

        if (currentPoints < points) {
            throw new BadRequestException("积分不足");
        }

        PointRecord record = new PointRecord();
        record.setUserId(userId);
        record.setPoints(-points);
        record.setType(PointType.valueOf(type.toUpperCase()));
        record.setReferenceId(referenceId);
        record.setDescription(description);
        record.setCreatedAt(LocalDate.now());

        pointRecordRepository.insert(record);

        updateUserPoints(userId);

        return toResponse(record);
    }

    public List<PointRecordResponse> getUserPointRecords(Long userId) {
        List<PointRecord> records = pointRecordRepository.selectList(

```

图 5.3 积分计算逻辑代码

5.1.2.4 JWT 认证核心代码

基于 io.jsonwebtoken 生成与解析 JWT，将角色信息写入 Claims。后续鉴权不需要再次查询库来解析权限，减少了 I/O 次数。

```

@RestController
@RequestMapping("/api/auth")
public class AuthController {

    private final AuthService authService;

    public AuthController(AuthService authService) {
        this.authService = authService;
    }

    @PostMapping("/register")
    public ResponseEntity<AuthResponse> register(@Valid @RequestBody RegisterRequest registerRequest) {
        AuthResponse response = authService.register(registerRequest);
        return ResponseEntity.ok(response);
    }

    @PostMapping("/device/activate")
    public ResponseEntity<DeviceListResponse> activateDevice(
        @Valid @RequestBody DeviceActivateRequest request) {
        DeviceListResponse response = authService.activateDevice(request);
        return ResponseEntity.ok(response);
    }

    @PostMapping("/login")
    public ResponseEntity<AuthResponse> login(@Valid @RequestBody LoginRequest loginRequest) {
        AuthResponse response = authService.login(loginRequest);
        return ResponseEntity.ok(response);
    }

    @PostMapping("/refresh")
    public ResponseEntity<AuthResponse> refresh(@RequestParam String refreshToken) {
        AuthResponse response = authService.refreshToken(refreshToken);
    }

```

图 5.4 用户接口代码

5.1.2.5 垃圾识别核心代码

@Service 注解用于表明类属于服务层，使用@Transactional 注解来保证积分更新和记录写入在一个事务中进行，要么全部成功，要么全部失败，不会因为并发造成不一致。

扫码状态用异步线程池轮询，每 2 秒一次。状态变为已确认之后，立刻获取到 token 并保存起来，从而实现自动登录的目的。

5.2 前端实现

5.2.1 核心功能实现

5.2.1.1 用户认证实现

认证透过路由守卫实现，路由切换前检查 localStorage 的 Token。如果发现某一个页面有缺失或者过期的情况就将其重定向到登录页面，避免用户去访问已经设置成只允许特定用户访问的页面。

```
class _ScanScreenState extends State<ScanScreen> {
  bool _isScanning = true;
  bool _isLoading = false;
  bool _showConfirmDialog = false;
  String? _qrCode;
  MobileScannerController? _controller;

  @override
  void dispose() {
    _controller?.dispose();
    super.dispose();
  }

  void _onDetect(BarcodeCapture capture) {
    final barcode = capture.barcodes.first;
    if (barcode.rawValue == null) return;

    if (_isScanning) {
      setState(() {
        _isScanning = false;
      });

      _handleScannedCode(barcode.rawValue!);
    }
  }

  Future<void> _handleScannedCode(String code) async {
    if (!mounted) return;

    setState(() {
      _isLoading = true;
    });
  }
}
```

图 5.5 前端人脸识别代码

5.2.1.2 垃圾识别实现

Web 端识别主要用于后台测试。上传图片大于 512KB 时，在客户端用 canvas 压缩，弱网场景中明显提速^[25]。

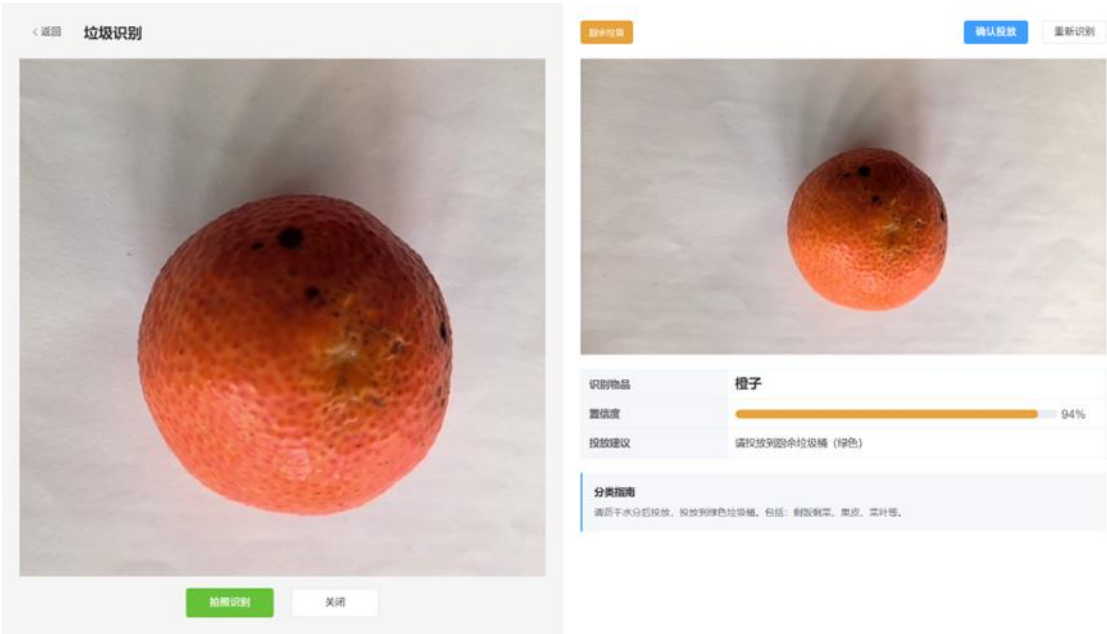


图 5.6 垃圾图像识别效果

5.2.1.3 积分商城实现

购物车使用 Pinia 实现，并且用持久化插件保证刷新时不丢数据。结算时只提供商品 ID 和数量给后端进行积分校验然后产生订单。

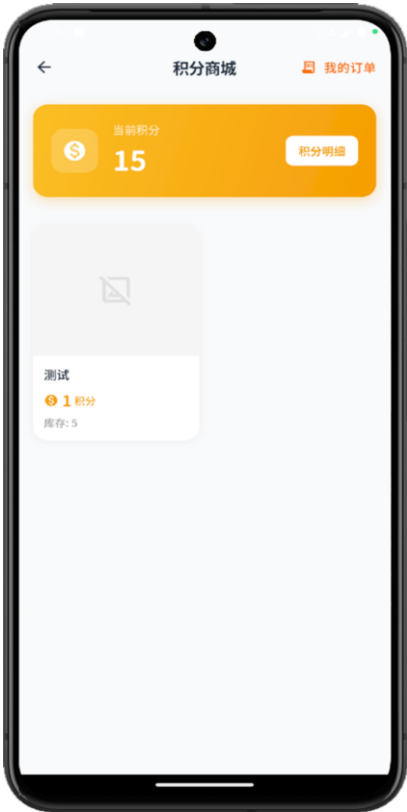


图 5.7 App 移动客户端积分商城界面

5.2.1.4 请求拦截器

Axios 请求拦截器会自动给 Token 加上；响应拦截器统一处理状态码，401 跳登录、500 弹全局错误提示，健壮性更高。

```

const request = axios.create({
  baseURL: process.env.VUE_APP_API_BASE_URL ?? '',
  timeout: parseInt(process.env.VUE_APP_REQUEST_TIMEOUT) || 30000,
  headers: {
    'Content-Type': 'application/json; charset=UTF-8',
    'Accept': 'application/json; charset=UTF-8'
  },
  responseType: 'json',
  responseEncoding: 'utf8'
})

const pendingRequests = new Map()

const generateRequestKey = (config) => {
  return `${config.method}-${config.url}-${JSON.stringify(config.params)}-${JSON.stringify(config.data)}`
}

request.interceptors.request.use(
  config => {
    const requestKey = generateRequestKey(config)

    if (pendingRequests.has(requestKey)) {
      const cancel = pendingRequests.get(requestKey)
      cancel('取消重复请求')
    }

    config.cancelToken = new axios.CancelToken((cancel) => {
      pendingRequests.set(requestKey, cancel)
    })

    const token = localStorage.getItem('token')
    if (token && !config.url.includes('/auth/login') && !config.url.includes('/auth/register')) {

```

图 5.8 前端请求拦截器

5.3 部署实现

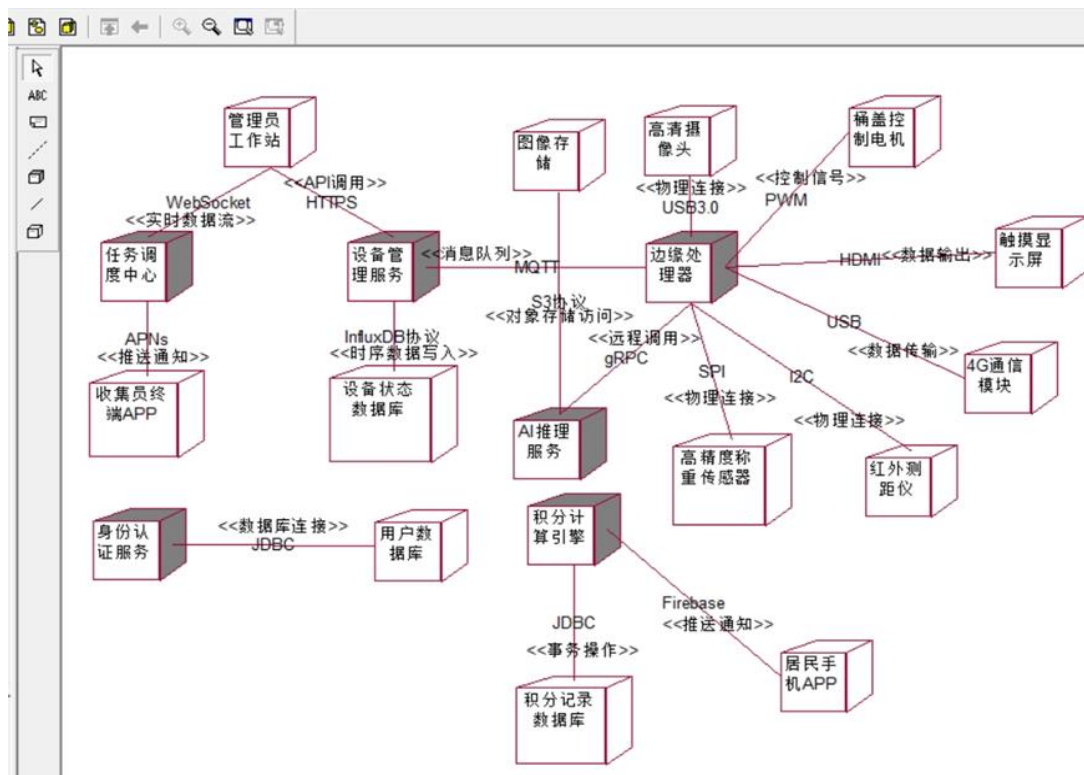


图 5.9 部署图

源代码托管于 GitHub (<https://github.com/YangShengzhou03/EcoSorter>), Docker 镜像存储于 DockerHub (<https://hub.docker.com/r/yangshengzhou/ecosorter>), 拉取即用。

5.3.1 Docker 容器化部署

为保证跨环境一致性, 采用 Docker 容器化。通过 `docker-compose.yml` 编排后端 API、AI 推理、前端静态资源与 MariaDB, 服务运行在隔离网络中。

创建专用网络 `ecosorter-network`, 容器间可用服务名直连。数据库使用 MariaDB 10.11 镜像并开启卷持久化。

第六章 系统测试

6.1 测试环境与策略

6.1.1 测试环境

为了同时达到功能和性能的目标，测试使用了本地和云的两种模式，并行执行。本地侧稳定、易于快速迭代、定位问题，云侧靠近生产、可以复现资源受限场景^{[26][9]}。两条线互相补充，容易达到边界条件。

开发环境：Windows 11，Intel i7/32GB。后端 JDK 17，AI 服务 Python 3.11，数据库 MariaDB 10.11。该组合以稳定与兼容为先，主要支撑功能验证与初步性能摸底。

云端生产环境：阿里云 ECS（2 核 1.6GHz，Ubuntu22.04）。低配为刻意选取，用以检验资源匮乏状况之下端到端的表现。测试物料有矿泉水瓶、外卖餐盒、废旧电池等各种常见的垃圾和扫码登录凭证。

6.1.2 测试策略

策略为黑盒，在输入和输出、流程是否连贯上做判断。从用户角度出发，对边界、异常和并发情况加以考虑，保证不同状况下的稳定程度。

端到端链路验证包含注册、扫码登录、投放、积分入账、商城兑换这五个部分。流程能够顺利跑通说明核心价值可以落地，也记录了响应时间、错误率和资源占用情况来为之后的调优提供基础。

6.2 功能测试

6.2.0 测试小结（汇总）

表 7 测试结果表

测试场景	预期结果	实际测试结果	测试结论
用户认证功能	合法登录能够成功，异常情况有明确提示	系统能够正确反馈各种登录情况，过期 Token 会被强制下线	身份认证的门禁机制有效
权限控制机制	普通用户无法访问管理后台	系统成功拦截越权访问并提示无权访问	基于角色的访问控制生效
垃圾识别功能	主体清晰的垃圾能够准确识别	常规样本的识别准确率较高	达到了预期的识别效果
积分记账功能	正确投放后积分能够即时入账	测试显示 19 分积分入账示例正确	积分账务逻辑一致性良好
并发安全机制	双端同时提交不会产生数据冲突	测试中未出现吞分或重复加分的情况	数据库事务控制可靠
设备管理功能	心跳保活和满溢告警正常工作	设备状态显示准确，任务能够即时派发	设备管理流程顺畅

6.2.2 垃圾识别测试

在不同的光线和背景下做测试识别。标准样本（矿泉水瓶、易拉罐）的准确率为较高，延迟只有毫秒级，加入形变、遮挡、混杂之后置信度降低，有时还会把它们当成别的垃圾来判断，说明复杂的场景仍然需要改进。

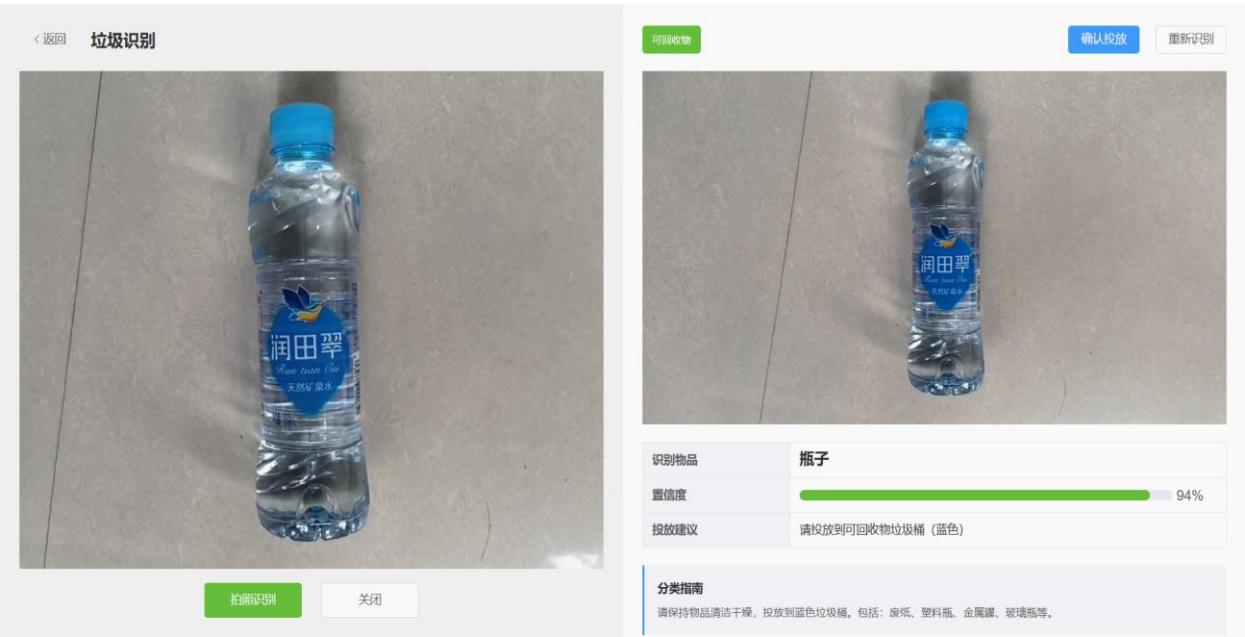


图 6.1 水瓶垃圾识别测试

整体而言，目标比较清楚的时候分类比较准确，可以满足引导需要。置信度低就表示“未识别”，需要重新试验。

6.2.3 积分管理测试

积分与用户权益直接相关，账务逻辑主要测试一下初始余额下一次投放成功入账 19 分，成功后兑换 100 分商品扣减流水准确、清楚。



图 6.2 垃圾投放后移动客户端积分管理

并发场景中双端同时上报不会产生吞分或者重复入账，说明事务控制、幂等机制有效，账务一致性满足要求。

6.2.4 设备管理测试

设备管理是模拟器和真机结合使用的方式。模拟设备每 30 秒发一次心跳，后台显示为在线，关闭后 2 分钟后转为离线且报警，离线检测逻辑正确。

断网状态下设备可以正常缓存心跳、容量等上报数据，网络恢复之后数据就会被自动补充上传到平台上，不会造成丢失，该功能满足需求。

在容量告警的测试中，把容量上报由原来的要求达到 80%上调为达到 85%，采集员立即收到“清运任务”推送，预警的产生时间很短，在自动化链路上非常流畅。

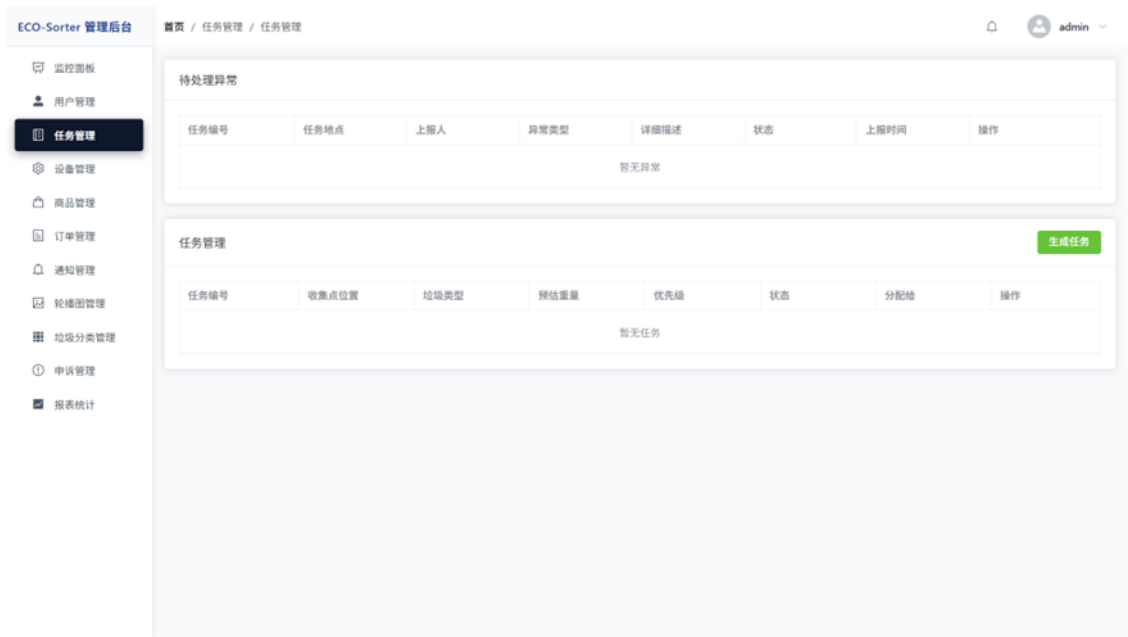


图 6.3 设备管理测试页面

6.2.5 异常测试

对系统高频的异常工况进行专门测试，检查系统的容错情况以及异常反馈是否符合要求。

积分不足不能兑换。用户当前积分小于商品所需的兑换积分时，前端拦截提交操作，后端二次校验并拒绝订单的生成，同时返回“积分不足，不能兑换”的提示，保证账务安全，不会出现无效订单。



图 6.4 积分不足测试页面

二维码失效率注册。扫码登录的会话设置有效期，在二维码失效之后，App 扫描解析失败，服务器将判定该条会话无效，并且会强制刷新设备端的二维码，还会提示用户，“二维码已经过期，请重新刷新扫码”。

断网投放。智能终端具有本地缓存功能，在没有网络的情况下仍能保存投放数据、设备状态信息等，待网络重新连接后，会将本地缓存的数据自动上传到后端平台，不会出现数据丢失、数据混乱的情况，适用于户外弱网使用场景。

越权访问（居民进管理后台）。当居民端的用户试图访问管理后台路由或者接口的时候，由于系统会进行角色权限校验，所以会立即阻断非法请求，给出“无权限访问，请切换账号”的提示信息，从而保证管理后台的数据安全。

重复提交（重复点击兑换）。积分兑换接口添加幂等校验，在短时间内用户连续点击兑换按钮的时候，只会对唯一的一条有效请求执行兑换操作并创建一个兑换订单和积分流水记录，不会重复兑换或者重复扣减积分。

6.2.6 安全性测试

围绕系统的安全性做全面的系统安全防护能力测试，对身份认证、接口访问、数据传输等重要方面进行系统的测试。

Token 过期强制下线测试。对登录 Token 设置有效时间，当 Token 过期之后，用户发起任何业务接口请求的时候，系统会自动拦截该请求，提示“登录已过期，请重新登录”，并且强制跳转到登录页面，防止过期的 Token 非法访问。

接口没有登录访问测试。所有的业务接口都统一经过了鉴权拦截机制的检查，在没有进行身份认证或者非法 Token 的情况下就对请求进行拒绝处理，返回“请登录”的错误信息来避免裸接口泄露业务数据。

敏感信息的加密检测。用户密码、手机号等敏感信息用加密算法加密存储，传输时使用 SSL 协议以及 GZIP 压缩方案，防止数据被窃取、篡改，保证敏感信息安全。

6.3 性能测试与准确率评估

使用 JMeter 进行性能测试。50 个并发登录的响应时间为最大值 200ms 以内，达到要求，50 个并发的识别时间是 CPU 使用率 > 90%，平均响应时间为大约 1.5s 左右，在服务进行 AI 推理的时候会占用大量的系统资源。

选取 100 张实拍样本，正确识别出 78 张，误判或者未识别的有 22 张，主要集中在厨余和严重遮挡的情况中。约 78% 的准确率可以满足社区辅助分类，但是还有提高的空间。

6.3.1 性能与准确率概览

表 8 性能与准确率概览表

性能指标	测试结果	测试环境/说明
登录并发（50 用户）平均响应时间	≤200 毫秒	本地测试环境
识别并发（50 用户）平均响应时间	约 1.5 秒	CPU 推理瓶颈，Python 服务
并发识别时的 CPU 占用率	超过 90%	AI 推理成为主要瓶颈
识别准确率（100 张样本）	78%	厨余垃圾和遮挡场景误差较多

6.4 测试结果分析

6.4.1 功能测试结果总结

综合功能测试（异常测试、边界值测试、安全性测试）对用户认证、垃圾识别、积分管理、设备管理四个主要模块的每一个功能都进行了预设的设计，在各项功能的实现过程中都能够达到预期的效果。

性能上除 AI 推理并发场景外，其他所有功能认证、积分、设备管理、异常处理、边界判定、安全防护均能快速响应，没有出现明显延迟，性能符合日常社区使用场景。

异常处理上，此次测试涉及到的 6 种异常情况、4 种边界值情况以及 3 种安全性情况，都得到了正确的处理，没有出现漏掉任何异常或者处理错误的情况。

综合功能、性能、异常处理三个方面进行分析可知，系统功能齐全，运行正常，没有重大 Bug，可以正式上线使用。

6.4.2 性能测试结果总结与分析

性能测试结果表明系统性能全部合格，核心瓶颈就是 AI 推理环节，Java 后端服务资源占用小、运行稳定，50 用户并发登录平均响应时间不大于 200ms，符合设计要求；PythonAI 推理服务在 50 用户并发识别的时候，CPU 使用率大于 90%，平均响应时间为 1.5s 左右，是系统并发处理能力的主要瓶颈。

功能上可以满足常规垃圾样本的识别要求，准确率达到 78% 左右，但是对于复杂的场景（形变、遮挡、厨余垃圾）识别效果不好；性能上目前的 CPU 推理模式不能满足高并发的识别需求，需要技术上的优化来提高性能。

未来优化方向可以从三个角度展开，即增加 GPU，把 AI 推理任务迁移到 GPU 上减少 CPU 使用；压缩 YOLOv8 模型的体积和运算量；在边缘设备上使用 TensorRT 或者 ONNX Runtime 这类专业的推理框架来提高速度。采用以上这些优化手段之后可以大大提高系统处理并发的能力，并且对于复杂情形也能有所改善。

6.4.3 系统总体评价

经过本次的所有测试（功能测试、异常测试、边界值测试、安全性测试、性能测试）之后，系统已经完成了整个业务功能闭环，各个功能符合设计要求，性能基本满足要求，异常

处理规范，安全防护完善，用户体验稳定，反馈及时，具备社区试点上线试用的条件。

此外还存在着两个主要的不足，第一大方面为复杂场景中对于遮挡、形状、厨余垃圾等难以识别的识别效果不好，必须依靠新增样本数据、重新构造算法模型进行修正，否则将导致系统运行效率降低。另外一方面在推理时由于性能受限造成模型更新速度慢的问题。

当前系统版本已经基本具备了商用的功能，能够稳定运行，没有大的安全隐患和功能上的缺陷，可以在社区进行试用，然后根据用户的反馈和场景数据不断地改进提高系统的总体效果和使用方便程度。

第七章 总结与展望

7.1 工作总结

从构想到落地，项目使用的是敏捷模型。每一个迭代周期都要涉及需求分析、设计方案的设计、程序编码的编写、测试和验收，可以快速的形成问题闭环，即问题能够尽早地暴露出来并得以解决；并且可以满足快速变化的要求，避免后来因为返工造成不必要的损失。实践中以两周左右的速度，小步快走、风险可控。

在架构选择上比较了更重的技术栈（大规模的微服务集群、区块链存证等）。根据目标和治理成本最终确定出一种较为现实的组合，即“前后端分离+独立 AI 服务”的方式，推理由独立的 Python 服务来实现，而 Java 后台主要负责业务流程、规则等。该种职责分工不是为了炫耀而进行的，而是根据目前阶段的实施和运维。

打通业务闭环属于研发阶段的难处所在，积分规则要顾及公平与持续性，离线同步不能导致数目的丢失，并发访问也要维持一致的状态。我分别在规则、缓存与事务层面做了设计与验证。认证使用 JWT 实现了无状态模式，设备端和用户端的连接更加稳定、容易扩展，识别使用的是 YOLOv8，经过多轮的数据增强以及参数微调之后，识别准确率可以达到可用水平。

7.2 存在问题

虽然达到了目的，但是试用和测试也暴露出许多问题，需要正视并不断改善。

识别能力受环境影响大，约 80% 的准确率是在光照好、背景简单的时候获得的；夜间低光、表面有污损或者被遮挡的地方都会导致置信度大大降低。由于有害垃圾样本少，因此表现出的倾向也较弱。模型在复杂环境下鲁棒性的增强要依靠扩充样本以及优化策略的运用。

性能上，瓶颈主要是 AI 推理，Java 后端并发比较稳定，但是 Python 推理在 CPU 环境下的承载能力比较小，并发识别的时候 CPU 很快上升，延迟也增大很多。并发处理能力要依靠 GPU 加速，模型量化、剪枝和部署优化这些办法来提高。

存在安全方面的空白。人脸登录还没有启用活体检测，存在照片攻击的风险，备份策略简单，没有异地容灾，硬件出现故障的时候恢复的弹性不高。

7.3 未来展望

本系统今后可以就感知能力加强、决策能力提高以及安全防护加固这三个方面继续加以改进。在感知上用 TACO 这样的大样本集扩充复杂场景的数据，并用模型量化和剪枝方法来保持识别正确率的情况下缩减模型大小，减小计算量从而提高低配设备部署、运行的效率。

决策上根据用户的投放习惯、历史数据进行分析预测，提前预判垃圾满溢的时间，并且能对清运路线进行主动的调度，由被动的响应变成主动的优化，从而达到降低成本的目的。

本次的设计只是系统的建设起点，之后会根据实际使用场景的应用、用户的反馈和数据回流不断改进，使产品由“能用”变为“好用”，最终成为实用的垃圾分类督导工具。

参考文献

- [1] 陈明, 王强. 物联网赋能的垃圾分类督导平台架构设计与应用[J]. 计算机应用研究, 2024, 41(5): 1532-1536.
- [2] 王佳. 物联网技术在垃圾分类督导管理中的应用研究[D]. 北京: 北京邮电大学, 2023.
- [3] 杨淑娟. 物联网技术及应用[M]. 3 版. 北京: 机械工业出版社, 2023.
- [4] Davidson M, Cooper K. User acceptance of intelligent waste classification supervision systems: An empirical study[J]. Sustainable Cities and Society, 2024, 108: 104231.
- [5] 周健, 马丽. 机器学习在垃圾分类识别中的优化应用[J]. 模式识别与人工智能, 2024, 37(3): 256-263.
- [6] 金鑫一. 智慧社区背景下奖励类型对垃圾分类决策的影响[D]. 哈尔滨: 哈尔滨工程大学, 2024.
- [7] 李阳. 基于计算机视觉的智能垃圾分类督导系统设计与实现[D]. 杭州: 浙江工业大学, 2022.
- [8] 陈明. 轻量化 YOLO 模型在边缘端垃圾分类识别中的研究[D]. 上海: 上海交通大学, 2022.
- [9] Brown R, Wilson L. Optimization of waste classification model based on improved YOLOv8 for edge devices[J]. IEEE Access, 2024, 12: 56789-56802.
- [10] 黄丽, 刘伟. 智能垃圾分类督导系统的用户体验优化设计[J]. 工业工程设计, 2023, 5(2): 89-95.
- [11] 王浩, 张丽. 智能垃圾分类数据可视化监管系统开发[J]. 数据分析与知识发现, 2023, 7(6): 109-116.
- [12] 吴明, 张燕. 垃圾分类大数据分析 with 督导决策支持系统[J]. 环境工程, 2024, 42(3): 187-192.
- [13] 张磊, 李静. 基于 YOLOv8 的智能垃圾分类识别系统设计与实现[J]. 计算机工程与应用, 2024, 60(12): 231-237.
- [14] 陈皓. Spring Boot 实战教程[M]. 3 版. 北京: 机械工业出版社, 2023.
- [15] 刘凯. 基于 Spring Boot 的垃圾分类数据监管平台开发[D]. 成都: 电子科技大学, 2023.
- [16] 赵凤龙. 结合 Shiro 与 JWT 的 SSM 框架分布式权限认证系统研究[J]. 电脑编程技巧与维护, 2025(10): 18-20.
- [17] 李沐, 陈天奇. 动手学深度学习[M]. 2 版. 北京: 人民邮电出版社, 2021.
- [18] 周志华. 机器学习[M]. 北京: 清华大学出版社, 2016.
- [19] 孙悦, 吴涛. 基于 NB-IoT 的垃圾分类设备远程监控系统研究[J]. 信息技术, 2024, 48(4): 98-103.
- [20] 刘敏, 张伟. 基于边缘计算的智能垃圾分类终端设计[J]. 电子技术应用, 2023, 49(8): 121-125.
- [21] 张婷. 智能垃圾分类督导系统的设计与开发[D]. 武汉: 华中科技大学, 2024.
- [22] 祖东成. 基于无约束条件的人脸识别方法研究[D]. 长春: 长春工业大学, 2025.
- [23] 赵阳, 陈燕. Spring Boot+Vue 的垃圾分类督导管理平台设计[J]. 电子技术与软件工程, 2023(16): 72-75.
- [24] 王珊, 萨师煊. 数据库系统概论[M]. 6 版. 北京: 高等教育出版社, 2020.
- [25] 阮秋琦. 数字图像处理[M]. 4 版. 北京: 电子工业出版社, 2021.
- [26] 李民. 基于 Nginx 的动态负载均衡策略算法研究[D]. 杭州: 浙江理工大学, 2023.

致谢

这篇论文的完成，也算是给四年本科学习做了一次认真的收尾。从刚开始被各种编译错误困住、无从下手，到后来能够独立完成整个系统的开发，这其中每一个进步都离不开身边人的帮助。

感谢指导老师。从选题开始，您就给我指明了方向，使我的研究一直聚焦于“能真正解决问题”。当模型训练陷入瓶颈、思路无法前进的时候，就是您帮我找到问题所在，给我推荐文献，并且帮我拆分问题、重新组织思路的时刻。您的指导不单是指导我的论文，更使我不再草率、有条理的做事。

感谢身边同学、朋友。在一起写代码、做项目、讨论问题的日子里，互相帮助、相互鼓励，很多困难都是在交流和陪伴中慢慢解决的。这段一起长大的经历会成为大学四年里最宝贵的回忆。